

Pub/Sub Fundamentals

A Hands-On Guide to Messaging, Fan-Out, and Delivery Guarantees

This guide walks through every core Pub/Sub capability — topics, schema validation, fan-out subscriptions, publishing, pull/push delivery, ack/nack, dead-letter topics, snapshots and replay, flow control, retry policies, filtering, and exactly-once delivery — using the Python **google-cloud-pubsub** client library throughout. It's meant to be read alongside the companion Colab notebook, or as a standalone print reference.

Problem statement: A retail platform's order system needs to notify several independent downstream systems the moment an order is placed — inventory, billing, analytics, fraud detection — without knowing or caring who's listening, and without waiting for any of them to finish. This guide builds up exactly the feature set needed to run that pattern reliably in production.

1. Setup — Authenticate & Configure

Authenticate with an account holding at least **Pub/Sub Editor**, enable the Pub/Sub API, then create both a publisher and subscriber client — every later operation reuses these two objects.

```
from google.cloud import pubsub_v1

publisher = pubsub_v1.PublisherClient()
subscriber = pubsub_v1.SubscriberClient()
```

Check it in the console: IAM & Admin > IAM → confirm your account has a Pub/Sub-covering role before proceeding.

2. Topics

The question: where do publishers actually send messages? A topic is the named channel publishers write to and subscriptions attach to — it holds no messages of its own once every subscription has consumed them.

```
topic_path = publisher.topic_path(PROJECT_ID, TOPIC_ID)
topic = publisher.create_topic(request={'name': topic_path})

# List every topic in the project
for t in publisher.list_topics(request={'project': f'projects/{PROJECT_ID}'}):
    print(t.name)

# Get details on one topic
fetched = publisher.get_topic(request={'topic': topic_path})
```

Check it in the console: Pub/Sub > Topics → your topic is listed, with a Metrics tab tracking publish volume over time.

3. Schema Registry — Contract-Driven Ingestion

The question: how do we stop a malformed message from ever reaching a subscriber, instead of discovering the problem downstream (e.g. a broken BigQuery load)? Schema Registry lets a topic reject messages that don't match an agreed shape at publish time.

```
schema_client = pubsub_v1.SchemaServiceClient()
schema = schema_client.create_schema(request={
    'parent': f'projects/{PROJECT_ID}',
    'schema': {'name': schema_path, 'type_': Schema.Type.AVRO, 'definition': avro_definition},
    'schema_id': SCHEMA_ID,
})

# Create a topic that enforces this schema
schema_topic = publisher.create_topic(request={
    'name': schema_topic_path,
    'schema_settings': {'schema': schema_path, 'encoding': Encoding.JSON},
})
```

The rejection happens server-side, at publish time — a malformed message never reaches any subscriber to discover the problem later. Publishing a payload missing a required field or with a wrong type raises an error on the publish call itself.

Check it in the console: Pub/Sub > Schemas → your schema and its Avro definition. Topic's Schema tab shows the attached schema and encoding.

4. Subscriptions — Pull, Push & Fan-Out

The question: how do multiple independent systems each get their own copy of every message? Every subscription attached to a topic receives an independent copy — acking on one subscription has zero effect on any other.

```
# Two independent pull subscriptions on the same topic
subscriber.create_subscription(request={'name': sub_a_path, 'topic': topic_path})
subscriber.create_subscription(request={'name': sub_b_path, 'topic': topic_path})

# A push subscription -- Pub/Sub calls your HTTPS endpoint instead of you polling
subscriber.create_subscription(request={
    'name': push_sub_path, 'topic': topic_path,
    'push_config': {'push_endpoint': PUSH_ENDPOINT_URL},
})
```

Pull vs. push: pull subscriptions are polled by your code (`subscriber.pull()` or streaming pull); push subscriptions have Pub/Sub call an HTTPS endpoint you own. A push subscription needs a real, verified endpoint (Cloud Run, Cloud Functions) to actually receive anything.

Check it in the console: Pub/Sub > Subscriptions → both subscriptions listed against the same parent topic. Topic's Subscriptions tab shows the same list scoped to that topic.

5. Publishing Messages

The question: how does a message actually get sent, including with metadata, in bulk, and — when order genuinely matters — in a guaranteed sequence?

```
# Single message with attributes
future = publisher.publish(topic_path, data=b'Hello!', source='training', priority='normal')
message_id = future.result()

# Batch -- publish() returns a future immediately, client batches under the hood
futures = [publisher.publish(topic_path, data=f'msg {i}'.encode()) for i in range(10)]
message_ids = [f.result() for f in futures]

# Ordered delivery -- requires an ordering-enabled publisher AND subscription
ordered_publisher = pubsub_v1.PublisherClient(
    publisher_options=pubsub_v1.types.PublisherOptions(enable_message_ordering=True))
subscriber.create_subscription(request={
    'name': ordered_sub_path, 'topic': topic_path, 'enable_message_ordering': True})
ordered_publisher.publish(topic_path, data=b'event', ordering_key='order-key-1')
```

Pub/Sub gives no ordering guarantee by default — messages sharing the same `ordering_key` are the only ones delivered in publish order. Ordering must be enabled on both the publisher client and the subscription for the guarantee to hold.

Check it in the console: Topic's Metrics tab → publish count increments with every call, single or batched, immediately visible.

6. Synchronous Pull & Acknowledge

The question: how does a consumer read messages and confirm it actually processed them — and what happens if it wants a message redelivered right away instead of waiting?

```
response = subscriber.pull(request={'subscription': sub_a_path, 'max_messages': 10})

# Acknowledge -- these will NOT be redelivered
ack_ids = [rm.ack_id for rm in response.received_messages]
subscriber.acknowledge(request={'subscription': sub_a_path, 'ack_ids': ack_ids})

# Negative-acknowledge (nack) -- forces immediate redelivery instead of
# waiting out the full ack deadline
subscriber.modify_ack_deadline(
    request={'subscription': sub_a_path, 'ack_ids': [nack_id], 'ack_deadline_seconds': 0})
```

Check it in the console: Subscription's Metrics tab → Unacked message count reflects the pull before acking, then drops to 0 after — direct proof the ack registered server-side.

7. Streaming Pull — Asynchronous Callback-Based Consumer

The question: polling with `pull()` in a loop works but is inefficient — what's the pattern most production consumers actually use? A long-lived stream that invokes your callback as messages arrive, no polling required.

```
def callback(message):
    print('Received:', message.data)
    message.ack()
```

```
streaming_future = subscriber.subscribe(sub_a_path, callback=callback)
# ... messages arrive and are acked automatically via the callback ...
streaming_future.cancel()
```

Check it in the console: Subscription's Metrics tab, watched live → Unacked message count rises as messages publish, then drops as the callback acks each one in real time.

8. Dead-Letter Topics

The question: a message keeps failing to process, no matter how many times it's redelivered — should Pub/Sub retry it forever? A dead-letter policy forwards a message to a separate topic after a maximum number of failed delivery attempts.

```
# The Pub/Sub service agent needs explicit permission to publish to your DLQ topic
# and to subscribe on your behalf for forwarding
# (gcloud pubsub topics add-iam-policy-binding ... --role=roles/pubsub.publisher)

subscriber.create_subscription(request={
    'name': dlq_source_sub_path, 'topic': topic_path,
    'dead_letter_policy': {
        'dead_letter_topic': dlq_topic_path,
        'max_delivery_attempts': 5,
    },
})
```

The IAM grant is easy to forget and a common reason dead-lettering silently doesn't work — both the dead-letter topic (needs Publisher role for the service agent) and the source subscription (needs Subscriber role for the service agent) require this explicit grant.

Check it in the console: Dead-letter topic's Permissions tab → Pub/Sub service agent listed with Publisher role. Source subscription's detail page shows the configured dead-letter policy and max attempts.

9. Snapshots & Replay

The question: we found a bug in our consumer's processing logic — can we replay messages we already consumed, once the fix is deployed? A snapshot freezes a subscription's ack state; seeking back to it makes everything unacked (or published after) redeliverable again.

```
subscriber.create_snapshot(request={'name': snapshot_path, 'subscription': sub_a_path})

# ... time passes, more messages are published and drained ...

# Replay: seek back to the snapshot
subscriber.seek(request={'subscription': sub_a_path, 'snapshot': snapshot_path})

# Or seek by timestamp directly, no snapshot needed
subscriber.seek(request={'subscription': sub_a_path, 'time': ten_minutes_ago_timestamp})
```

Check it in the console: Pub/Sub > Snapshots → your snapshot listed with its source subscription and auto-expiration date (7 days by default if unused).

10. Flow Control & Retry Policy

The question: how do we stop a fast publisher from overwhelming a slow consumer, and control how long Pub/Sub waits before redelivering an unacked message?

```
# Client-side flow control -- caps in-flight unacked messages/bytes
flow_control = pubsub_v1.types.FlowControl(max_messages=5, max_bytes=1024*1024)
subscriber.subscribe(sub_b_path, callback=my_callback, flow_control=flow_control)

# Server-side retry (backoff) policy -- no client code needed to benefit
subscriber.create_subscription(request={
```

```

    'name': retry_sub_path, 'topic': topic_path,
    'retry_policy': {
        'minimum_backoff': {'seconds': 10},
        'maximum_backoff': {'seconds': 600},
    },
})

```

Check it in the console: Retry-policy subscription's detail page → Retry policy field shows the configured min/max backoff values directly.

11. Message Filtering

The question: can a subscription receive only messages relevant to it, without pulling everything and filtering in application code? A subscription-level filter matches on message attributes server-side — non-matching messages are auto-acknowledged and never seen.

```

subscriber.create_subscription(request={
    'name': filtered_sub_path, 'topic': topic_path,
    'filter': 'attributes.priority = "high"',
})

publisher.publish(topic_path, data=b'low priority event', priority='low').result()
publisher.publish(topic_path, data=b'high priority event', priority='high').result()
# Pulling this subscription returns only the high-priority message

```

Check it in the console: Filtered subscription's detail page → Filter field shows the exact expression. Its Metrics tab's Unacked message count reflects only matching messages.

12. Exactly-Once Delivery

The question: Pub/Sub's default at-least-once delivery means a message might occasionally be redelivered after you've already acked it — can we get a stronger guarantee?

```

subscriber.create_subscription(request={
    'name': exactly_once_sub_path, 'topic': topic_path,
    'enable_exactly_once_delivery': True,
})

ack_response = subscriber.acknowledge(
    request={'subscription': exactly_once_sub_path, 'ack_ids': [ack_id]})
# A successful ack response is now a guarantee, not a best-effort signal

```

This is a newer subscription feature with its own throughput/latency trade-offs — check current GCP documentation before recommending it for a high-throughput production pipeline.

Check it in the console: Exactly-once subscription's detail page → Exactly-once delivery field reads Enabled.

13. Cleanup

Delete every subscription, the snapshot, every topic, and the schema, in that order — run this last, once every demo above has been shown.

```
for sub_id in all_subscription_ids:
    subscriber.delete_subscription(
        request={'subscription': subscriber.subscription_path(PROJECT_ID, sub_id)})

subscriber.delete_snapshot(request={'snapshot': snapshot_path})

for t_path in [topic_path, schema_topic_path, dlq_topic_path]:
    publisher.delete_topic(request={'topic': t_path})

schema_client.delete_schema(request={'name': schema_path})
```

Check it in the console: Pub/Sub > Topics, Subscriptions, and Snapshots → none of this notebook's training-... prefixed resources should remain listed.

This guide's companion notebook: everything above has a fully executable, self-contained Colab/Jupyter notebook counterpart following this exact sequence, with live console checkpoints and a final cleanup-verification step at every stage.